

Data Detective – Audit 4

**Entwicklungsprojekt WS 25/26
von Florian Roß und Karim Khemiri**

Inhaltsverzeichnis

1. Erneuter Überblick POCs (Florian)
 2. Achievements und Daily Challenges (Florian)
 3. Persönliche Statistiken (Florian)
 4. Extra Schwierigkeitsgrade und neue Methoden (Florian)
 5. Main Prototype Life Präsentation (Karim)
 6. Generelle Improvements (Karim)
 7. Auswertung des Fachgesprächs (Karim)
 8. Derzeitiger Projektstand, Ausblick und Herausforderungen (Karim)
- (Quellen und Kontakt zum Fachgesprächspartner)

Durchgeführte PoCs

PoC 107 Stilistische Manipulationen ✓

Es soll geprüft werden, ob Diagramme über rein visuelle Stilmittel (Fehlende Beschriftungen, Style(Farben, strichstärken, hervorhebungen etc.), Overlaps / Occlusion) manipuliert werden können.

- Zwei Neue Manipulationstypen (Balkenverzerrung/Farb-Highlighting)

PoC 109 Reale Anwendungsbeispiele zu Statistik-Manipulationen ✓

Es soll geprüft werden, ob nach Lösungsabgabe passende hochwertige Realbeispiele bereitgestellt werden

- Funktioneller Historisches BeispielButton (+Einbindung Newsscreen)

PoC 110 Gamification ✓

Es soll geprüft werden, ob die App genügend Spielspaß und Erfolgserlebnisse liefert, zur Gewährleistung der Nutzermotivation.

- Achievements,
- Daily Challenges,
- 2 neue Spielmodi(Double und Survial),
- PersonalScreen(Stats, Streaks, MasteryChart)

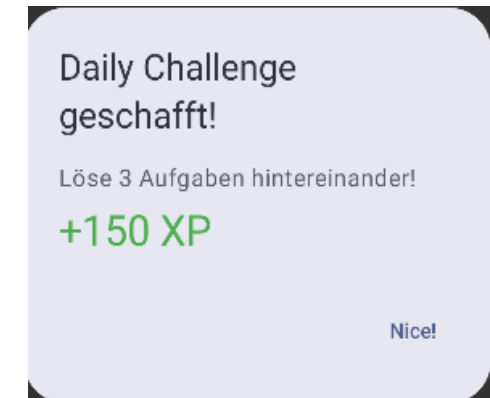
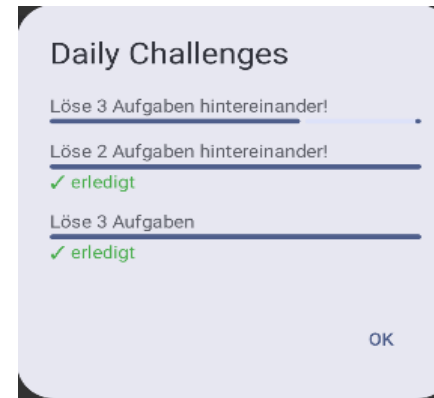
PoC 112 Persistenz Spiel- und Lernfortschritt ✓

Es soll geprüft werden, ob Nutzerfortschritt zuverlässig gespeichert und geladen werden kann (XP, Level, personelle Statistik, Achievements)

Achievements und Daily Challenges

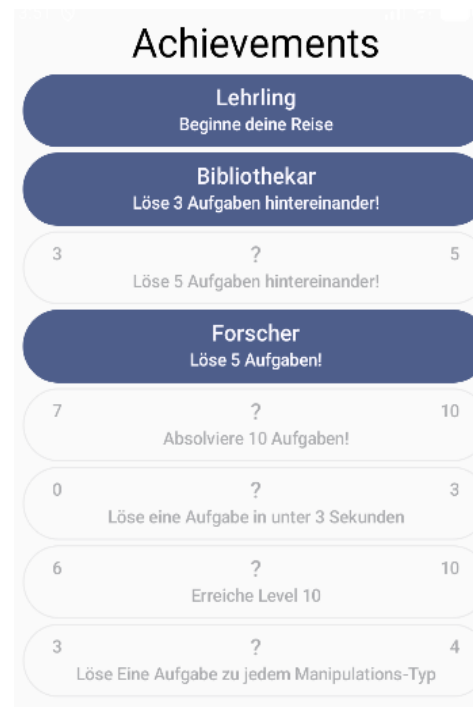
Challenges/
Achievements als
Datenobjekte mit
definierten
Conditions

```
data class DailyChallenge (  
    val id: String,  
    val hint: String,  
    val condition: DailyCondition,  
    val required: Int,  
    val xp: Int,  
)
```



Challenges resetten und rotieren
automatisch täglich

```
val today = LocalDate.now().toString()  
val storedDate = prefs[PreferencesKeys.DAILY_DATE]  
  
if (storedDate != today) {  
    prefs[PreferencesKeys.DAILY_DATE] = today  
    prefs[PreferencesKeys.DAILY_SOLVED] = 0  
    prefs[PreferencesKeys.DAILY_DONE] = 0  
    prefs[PreferencesKeys.DAILY_STREAK] = 0  
    prefs[PreferencesKeys.DAILY_BEST] = 0  
    prefs[PreferencesKeys.DAILY_COMPLETED] = ""  
  
    ManipulationType.entries.forEach {  
        prefs[PreferencesKeys.dailySolvedKey(type = it)] = 0  
    }  
}
```



Persistente Speicherung des Fortschritts lokal über
DataStore (Nach jeder Aufgabe)

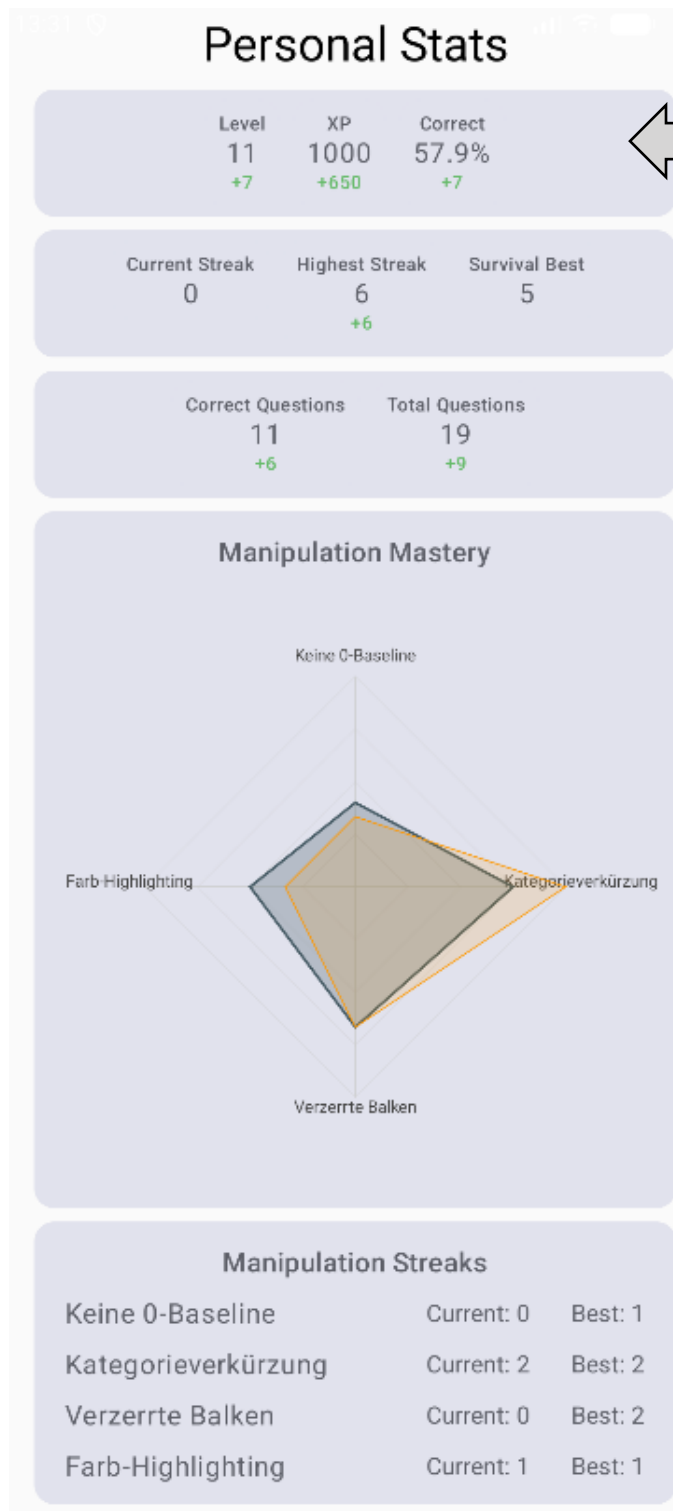
```
if (correctAnswer) {  
    // Stats  
    val solved = (prefs[PreferencesKeys.SOLVED_QUESTIONS] ?: 0) + 1  
    prefs[PreferencesKeys.SOLVED_QUESTIONS] = solved  
}
```

Bsp: Gesamt gelöste Aufgabenanzahl

Daily Challenge Pop-Up bei Erfüllung

```
todayChallenges.forEach { challenge ->  
  
    val progress = challenge.condition.currentValue(profile)  
    if (progress >= challenge.required &&  
        challenge.id !in completedSet) {  
        completedSet = completedSet + challenge.id  
        popup = challenge  
    }  
}
```

Persönliche Statistiken



Speicherung der allgemeinen Statistiken über die PreferencesKeys die auch für die Achievements genutzt werden

Mastery Graph

```
// Mastery Linie
val masteryPath = Path()
types.forEachIndexed { index, type ->
    val value = mastery[type]?.coerceIn(0f, 1f) ?: 0f
    val angle = angleStep * index - Math.PI.toFloat() / 2
    val r = radius * value
    val x = center.x + r * cos(x = angle)
    val y = center.y + r * sin(x = angle)
    if (index == 0) masteryPath.moveTo(x, y)
    else masteryPath.lineTo(x, y)
}
masteryPath.close()
```

Hilfsfunktion, die Richtig-Anzahl pro Manipulation trackt

```
fun masteryByManipulation(
    solved: Map<ManipulationType, Int>,
    failed: Map<ManipulationType, Int>
): Map<ManipulationType, Float> {
    return ManipulationType.entries.associateWith { type ->
        val s = solved[type] ?: 0
        val f = failed[type] ?: 0
        val total = s + f
        if (total == 0) 0f else s.toFloat() / total
    }
```

Snapshot alle 10 Aufgaben für Entwicklungsvergleich

```
// Snapshot alle Paar Aufgaben
if (done % 10 == 0) {
    prefs[PreferencesKeys.SNAP_XP] = prefs[PreferencesKeys.XP] ?: 0
    prefs[PreferencesKeys.SNAP_LEVEL] = prefs[PreferencesKeys.LEVEL] ?: 1
    prefs[PreferencesKeys.SNAP_SOLVED] = prefs[PreferencesKeys.SOLVED_QUESTIONS] ?: 0
    prefs[PreferencesKeys.SNAP_DONE] = prefs[PreferencesKeys.DONE_QUESTIONS] ?: 0
    prefs[PreferencesKeys.SNAP_HIGHEST_STREAK] = prefs[PreferencesKeys.HIGHEST_STREAK] ?: 0
    ManipulationType.entries.forEach { type ->
        prefs[PreferencesKeys.snapSolvedKey(type)] = prefs[PreferencesKeys.solvedKey(type)] ?: 0
    }
    ManipulationType.entries.forEach { type ->
        prefs[PreferencesKeys.snapFailedKey(type)] = prefs[PreferencesKeys.failedKey(type)] ?: 0
    }
}
```

Streaks pro Manipulation

```
//Zähler für jeweiligen Manipulationstypen
manipulationType?.let { type ->
    val key =
        if (correctAnswer)
            PreferencesKeys.solvedKey(type)
        else
            PreferencesKeys.failedKey(type)
    prefs[key] = (prefs[key] ?: 0) + 1
}
//Streaks für einzelne Manipulationstypen erhöhen
manipulationType?.let { type ->
    val streakKey = PreferencesKeys.streakKey(type)
    val bestKey = PreferencesKeys.bestStreakKey(type)
    if (correctAnswer) {
        val newStreak = (prefs[streakKey] ?: 0) + 1
        prefs[streakKey] = newStreak

        val best = prefs[bestKey] ?: 0
        if (newStreak > best) prefs[bestKey] = newStreak
    } else {
        prefs[streakKey] = 0
    }
}
```

Extra Schwierigkeitsgrade und neue Methoden

Balkenverzerrung:

Berechnung des Intensitätswert in
ManipulatedChart

```
val targetDistortion =  
    if (!showCorrect && distortion != null)  
        distortion.intensity  
    else  
        1f
```

Verzerrung in Chart Composable mit zuvor
errechnetem distortionFactor

```
//Verzerrung der Balkenhöhe bei DISTORTED_BAR_LENGTH  
val relativeHeight = normalized.pow(x = distortionFactor)  
val barHeight = relativeHeight * chartHeight  
val yTop = paddingTop + chartHeight - barHeight
```

Farb-Highlighting:

Alle bis auf ein zufälliger Balken/Punkt
bekommen eine leichte Transparenz

```
if (!highlightEnabled || highlightedIndex == null) {  
    return baseColor  
}  
return if (index == highlightedIndex) {  
    baseColor //volle Farbe  
} else {  
    baseColor.copy(alpha = 0.85f) //leicht transparent  
}
```

Double und Survival Modi

```
enum class ManipulationMode {  
    9 Usages  
    SINGLE,  
    6 Usages  
    DOUBLE,  
    10 Usages  
    SURVIVAL  
}
```

Manipulationen im Task werden
nun als Liste gespeichert, damit
Taskgenerator auch Double-
aufgaben generieren kann

```
val manipulations: List<Manipulation>,
```

Survival Modus

```
//Survivalmode logik  
if (manipulationMode == ManipulationMode.SURVIVAL) {  
    if (isCorrect) {  
        survivalStreak++  
        if (survivalStreak > survivalBest) {  
            survivalBest = survivalStreak  
            // Persistieren im DataStore  
            viewModelScope.launch {  
                userData.saveSurvivalBest(survivalBest)  
            }  
        }  
    } else {  
        survivalRunResult = SurvivalResult(  
            streak = survivalStreak,  
            best = survivalBest  
        )  
        survivalStreak = 0  
        return  
    }  
}
```

Run beendet

Aufgaben geschafft

2

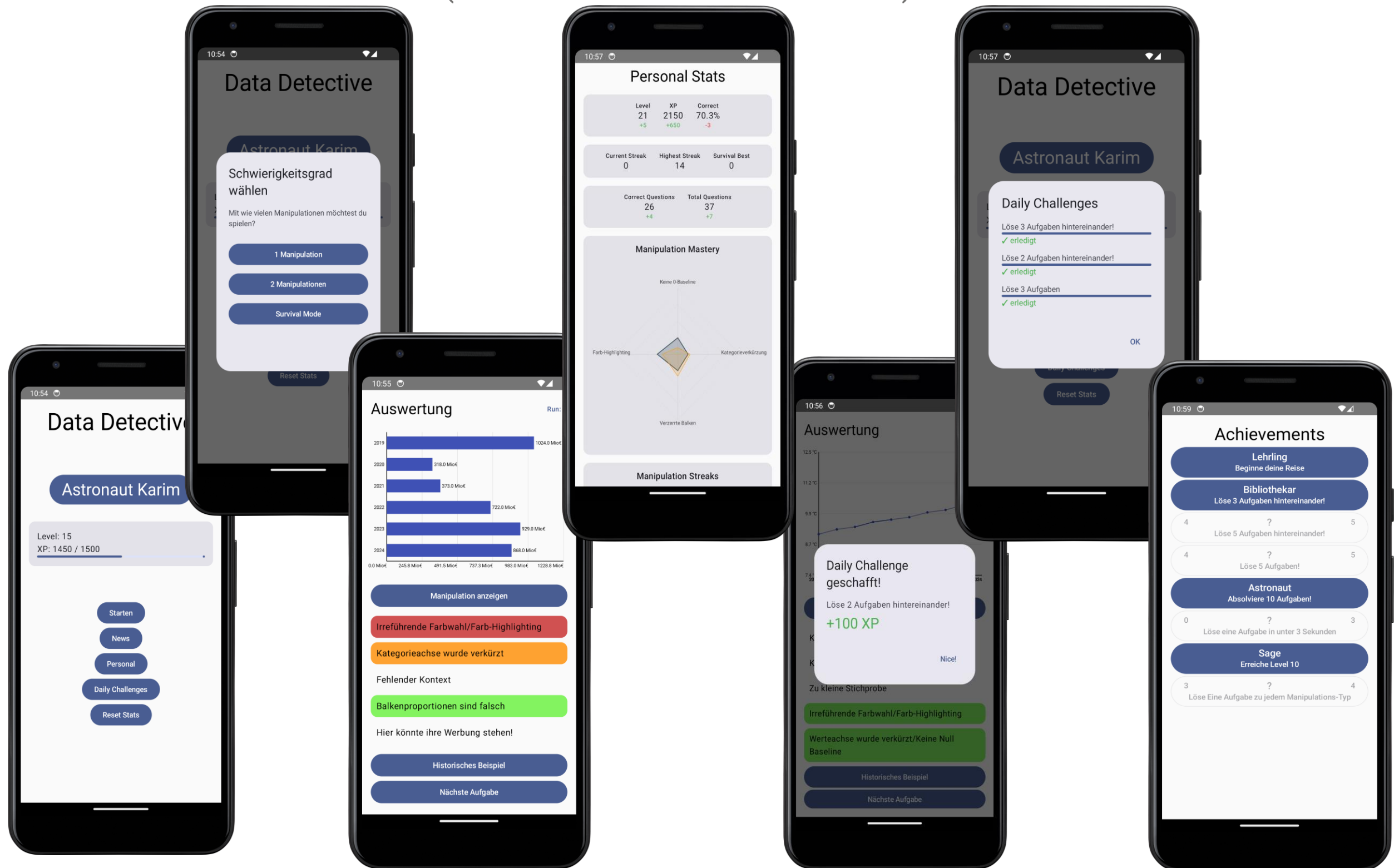
Bester Run

6

Zurück zum Menü

Data Detective Rapid Prototype

(Placeholder für den Foliensatz)



Generelle Improvements

Bugfixes:

- Statistiken und dessen Animationen glitchen nicht mehr beim Wechsel der Screens
- Alle Screens nun korrekt an Homescreen gebunden
- Alle Fade ins und Fade outs funktionieren nun einwandfrei

UI:

- Konsistentere und kompaktere Screen-Darstellungen
- Run Counter als Info für den Nutzer

Allgemein:

- Nutzer muss nun die richtige Aufgabenauswahl auswählen und wird über Schwierigkeitsgrad hingewiesen
- Run Counter zählt beendete Aufgaben der Session
- Persistente Speicherung aller Nutzerdaten
- Zurücksetzen der Nutzerdaten (Mit Schutz)
- Nutzer wird über Daily Challenges und erlangte XP durch Pop-Up hingewiesen
- "News"-Tab würde überarbeitet und werden nun genutzt als Real-Beispiele innerhalb der Aufgaben

Gamification/MCI-Fachgespräch mit Simon Ludwig Schulte

Reines visuelles Feedback und Progress tracking unzureichend, Belohnungen ohne klare Sinnhaftigkeit eher wertlos

Der Personal Screen:

- Top Scores und besonders Streak Counter als starke Motivationsquelle
- Durch Zahlen und eigener Statistik vielseitig ansprechend

Historische Beispiele:

- Dienen als gutes Tool zur Entwicklung von Rahmenbedingungen für zusätzliches Interesse intrinsischer Motivation
- Binden Interesse und emotionales Investment an Kompetenz und Lernerfolg

Achievements:

- laut Feedback immer noch zu wenig Kompetenz/Skill-basiert
- Benötigen stärkere Belohnungen, Ziele oder andere Sinnhaftigkeit statt reines Completionist-Prinzip und Titel

Daily Challenges und Level System:

- Visuelles reines Progress-Tracking keine ausreichende Motivationsquelle, besitzt im Gegensatz zu Achievements keine wirkliche Belohnung, ausbesserbar durch Kauf-, Tausch-Mechaniken oder Customization

-> **Empfehlungen:** Noch stärkerer Fokus auf Streaks (Umgesetzt), XP/Level Ausbau, Survival Mode als Spielmodus orientiert an Streaks (umgesetzt), Freischaltbarer Content, RAMP- als Gamification Guideline, Verknüpfung der einzelnen Systeme

Derzeitiger Projektstand, Ausblick und Herausforderungen

Stand:

- Zielerreichungsgrad: Hoch, Konzept wurde voll in vertikalem Prototyp umgesetzt
- Deutliche Ausarbeitung der Grundlegenden Funktionen und der Gamification
- Massiver Sprung in Code Qualität von Audit 3 zu Audit 4
- Bestätigung das genutzte Bibliotheken und die Kompetenz ausreichen um Projekt weiter zu führen

Ausblick:

- Viele Möglichkeiten zum Ausbau: Mehr Schwierigkeitsgrade, Kategorien, (Online-Mode)
- Verbesserte Gamification auf Basis des Fachgesprächs
- Fokus für Audit 4 war Gamification deshalb zukünftig eher wieder auf Methoden als Fokus zur Implementierung
- Nutzertests

Herausforderungen:

- Passende Realbeispiele finden schwieriger als erwartet
- Ob alle Aufgabentypen / Manipulationsmethoden im Rahmen der App relevanten Lernerfolg bieten unschlüssig
- "Künstlich" angepasste Statistiken weiterhin ein Problem, sehr Methodenabhängig

Quelle zum RAMP-Prinzip und Verweis zum Fachgesprächspartner

https://www.th-koeln.de/personen/simon_ludwig.schulte/

<https://www.gamified.uk/gamification-framework/the-intrinsic-motivation-ramp/>